

A Deep Recurrent Survival Model for Unbiased Ranking

Jiarui Jin¹, Yuchen Fang¹, Weinan Zhang¹, Kan Ren², Guorui Zhou³, Jian Xu³, Yong Yu¹,
Jun Wang⁴, Xiaoqiang Zhu³, Kun Gai³

¹Shanghai Jiao Tong University, ²Microsoft Research, ³Alibaba Group, ⁴University College London
{jinjiarui97, arthur_fyc, wnzhang, yyu}@sjtu.edu.cn, kan.ren@microsoft.com, jun.wang@cs.ucl.ac.uk, {guorui.xgr, xiyu.xj, xiaoqiang.zxq, jingshi.gk}@alibaba-inc.com

ABSTRACT

Position bias is a critical problem in information retrieval when dealing with implicit yet biased user feedback data. Unbiased ranking methods typically rely on causality models and debias the user feedback through inverse propensity weighting. While practical, these methods still suffer from two major problems. First, when inferring a user click, the impact of the contextual information, such as documents that have been examined, is often ignored. Second, only the position bias is considered but other issues resulted from user browsing behaviors are overlooked. In this paper, we propose an end-to-end Deep Recurrent Survival Ranking (DRSR), a unified framework to jointly model user's various behaviors, to (i) consider the rich contextual information in the ranking list; and (ii) address the hidden issues underlying user behaviors, *i.e.*, to mine observe pattern in queries without any click (non-click queries), and to model tracking logs which cannot truly reflect the user browsing intents (untrusted observation). Specifically, we adopt a recurrent neural network to model the contextual information and estimates the conditional likelihood of user feedback at each position. We then incorporate survival analysis techniques with the probability chain rule to mathematically recover the unbiased joint probability of one user's various behaviors. DRSR can be easily incorporated with both point-wise and pair-wise learning objectives. The extensive experiments over two large-scale industrial datasets demonstrate the significant performance gains of our model comparing with the state-of-the-arts.

CCS CONCEPTS

• Information systems → Learning to rank.

KEYWORDS

Unbiased Learning-to-Rank; Position Bias; Cascade Model

ACM Reference Format:

Jiarui Jin, Yuchen Fang, Weinan Zhang, Kan Ren, Guorui Zhou, Jian Xu, Yong Yu, Jun Wang, Xiaoqiang Zhu, Kun Gai. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20)*, July 25–30, 2020, Virtual Event, China. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3397271.3401073>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGIR '20, July 25–30, 2020, Virtual Event, China

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-8016-4/20/07...\$15.00

<https://doi.org/10.1145/3397271.3401073>

1 INTRODUCTION

Nowadays, information systems have become a core part for the personalized online services, such as search engines and recommender systems, where machine learning is the key technique for the success [3]. Among them, learning-to-rank [29] is a fundamental approach which learns to present a ranked list of items as the most relevant to the user query or most likely preferred by the target user. However, there is no ground truth except expensive labeled data from human experts for training a ranker, which facilitates the usage of implicit user feedbacks [20, 42], *i.e.*, user clicks [21, 42] and browsing logs [2, 42]. Although the usage of the implicit user feedback alleviates data labeling cost, it introduces the data bias problem [2, 21]. As shown in Figure 1, taking position bias as an example, a user typically observes the presented item list from top to down, in such a way the attention of the user drops rapidly and the user may observe and click more likely on the top presented items than the bottom ones [20]. Simply optimization for the ranking performance based on the implicit feedback data may result the ranking function in learning the presenting order, rather than the true relevance or the real user preferences. To tackle such a bias issue, many researchers have explored the potential technical approaches in training a high efficient model with unbiased learning-to-rank.

One line of the research is based on counterfactual learning [1, 18], which treats the click bias as the counterfactual factor [35] and debiases the user feedbacks through inverse propensity weighting (IPW) [21, 42]. For instance, Ai et al. [2] and Hu et al. [17] respectively proposed to employ the dual learning method for jointly estimating position bias and training a ranker. However, these methods either focus on the position of the item while ignoring the **contextual information** of the given ranking list, *e.g.*, the content of the previous items may influence the observation of the next item [2, 17], or optimize the ranking performance directly while neglecting the nature of the user browsing behaviors, *e.g.*, the click always happens at the observed item [21, 42]. Another line of research investigates user browsing behavior model [7, 10, 38, 39], where several basic assumptions about the user browsing behaviors are adopted to maximize the likelihood of the observations. For example, Fang et al. [12] extended position-based model and proposed an estimator based on invention harvesting, which, however, only considers the query-level contents (*e.g.*, query feature) rather than the document-level contextual information (*e.g.*, the previous observed documents).

More importantly, the prior works often focus on addressing the gap between user behavior and true preferences, while leaving latent issues hidden in the user behaviors, to be unsolved: (i) As Figure 1 illustrated, when the user starts a search with a query, she

may stop browsing by interruption or end the search session due to lack of interest, which leaves many queries without any user click behavior, often referred as **non-click queries**. It is impractical to mine click patterns in these non-click queries. However, these queries contain large amount of observe patterns, e.g., users are often impatient and only observe several top documents when selecting daily items such as fruits; while becoming patient and browse several sessions before click when selecting luxury items such as phones. Different from recent investigations on abandoned queries, defined as Web search queries without hyperlink clicks, we here mainly focus on mining observe patterns instead of distinguishing bad and good abandonment [27, 36]. (ii) When the user scrolls down the page and observes the presented items, the system may track (through check points) that the user has observed until the last position of the screen. However, this may not be true since the user would have stopped and lost her attention before checking the items on the last positions. It is not realistic to set eyetracking for each user during each of her visit [20]. Hence, the tracking logs of the user browsing history cannot truly tell that the users are actually checking the contents, and we called these noisy logs **untrusted observations**, as Figure 1 shows.

Based on the above analysis, when designing unbiased learning-to-rank algorithm, the current state-of-the-art methods have not well solved, even may not been aware of, the following challenges, which we address in this paper: **(C1)** The user behaviors contain various and highly correlated patterns based on the **contextual information**. **(C2)** There are large scale of latent observe patterns hidden in the **non-click queries**. **(C3)** The **untrusted observation**, another unsolved issue, is caused by limitation of tracking logs.

To tackle these challenges, we propose a novel framework called *deep recurrent survival ranking* (DRSR) to formulate the unbiased learning-to-rank task as to estimate the probability distribution of user's conditional click rate. To capture user behavior pattern, we combine survival model and recurrent neural network (RNN) in DRSR framework. Specifically, the RNN architecture incorporates all the top-down contents in the ranking list as contextual information, while the survival model derives the joint probability of user behavior via the probability chain rule, which enables modeling both observed and censored user behavior patterns **(C1)**. We then assume that a user's favored documents in the non-click queries could hide in unobserved ones out of browsing scope. This is similar to those patients who leave the hospital and die out of investigation period. Hence, we can leverage survival analysis techniques [8, 22?] via treating non-click logs as censored data of clicked ones where the censorship occurs in the click behavior **(C2)**. In seeking a proper way to measure relevance for untrusted observation, we model conditional probability and design a novel objective function to learn relative relevance between trusted and untrusted feedbacks in pair-wise setting **(C3)**.

The major contributions of this paper can be outlined as follows.

- We propose an innovative framework to jointly capture the correlation of user behaviors and train an unbiased rank with contextual information of the rank list.
- We incorporate cascade model with survival analysis to deeply mine hidden user observe patterns in non-click queries.

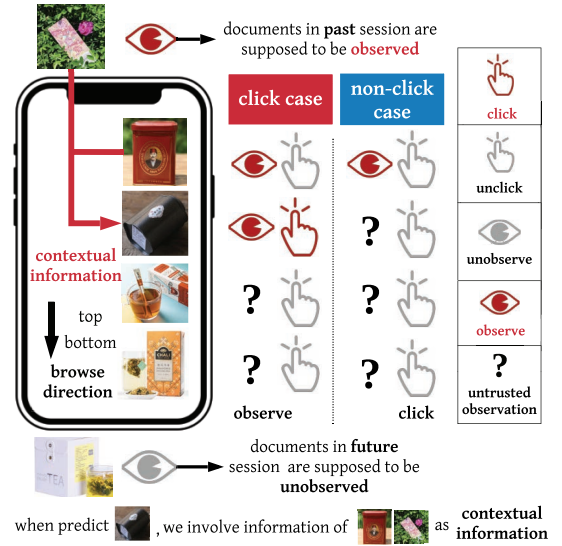


Figure 1: Illustration of user various behaviors (i.e., click and non-click case) when browsing document list as shown in the left side. Notations are provided in the right side.

- We provide a Pairwise Debiasing training scheme to model relative relevance between trusted and untrusted observations.

Extensive experiments on Yahoo search engine and Alibaba recommender system datasets demonstrate the superiority of DRSR over state-of-the-arts. To the best of our knowledge, in the unbiased learning-to-rank task, it is the first work providing adaptive user behavior modeling using contextual information with survival analysis.

2 RELATED WORK

Unbiased Learning to Rank. Learning to rank [29] is a fundamental technique for information systems, such as search engine, recommender system and sponsored search advertising. There are two streams of unbiased learning to rank methodologies. One school is based on some basic assumptions about the user browsing behaviors [7, 10, 38, 39]. These models maximize the likelihood of the observations in the history data collected from the user browsing logs. Recently, Fang et al. [12] extended position-based model and proposed an effective estimator based on invention harvesting. As is discussed in [21], these model only model user behavior patterns without sufficient optimization for learning to rank problem. The other school derived from counterfactual learning [21, 43] which treats the click bias as the counterfactual factor [35] and debias the user feedback through inverse propensity weighting [42]. Recently, Ai et al. [2] and Hu et al. [17] respectively proposed to employ the dual learning method for jointly estimating position bias and training a ranker. However, these prior works often ignore the rich contextual information in query and omit user's various behaviors except click. In this paper, we propose an innovative approach a novel cascade model adaptive in both point-wise and pair-wise setting. In addition to taking joint consideration of click and non-click data via survival analysis, we also model the whole ranking list through recurrent neural network.

Table 1: A summary of notations in this paper.

$q, D_q, C_q, O_q, \mathcal{D}$	Query q and set of documents D_q associated with click information C_q and observe information O_q . Dataset $\mathcal{D}$ for all query, formulated as $\mathcal{D} = \{(q, C_q, O_q, D_q)\}$.
$i, d_i, \mathbf{x}_i, r_i, c_i, o_i$	Position i and i -th document d_i in D_q with feature vector $\mathbf{x}_i$, relevance information r_i , click information c_i and observe information o_i . Note that we utilize binary value here and can obtain $o_i = 1$ from $c_i = 1$.
$p_i, h_i, W(i), S(i)$	At i -th document d_i , <i>click probability</i> (P.D.F) p_i and <i>relevance probability</i> (conditional probability) h_i as defined in Eqs. (7). and (8). <i>observe probability</i> (C.D.F.) $S(i)$ and <i>unobserve probability</i> (C.D.F.) $W(i)$ as defined in Eqs. (5) and (6).
D_q^+, D_q^-, D_q^*, I_q	Real positive document set D_q^+ document set D_q^- and uncertain document set D_q^* the whole document set D_q formulated as $D_q = D_q^+ \cup D_q^- \cup D_q^*$, I_q denotes the set of document pairs (d_i, d_j) . where d_i, d_j are sampled from two sets of D_q^+, D_q^-, D_q^* .

Survival Analysis. In the field of learning-to-rank, deep learning-based sequential algorithms have won much attention and surpassed several traditional models via considering the time information of each interaction [11]. However, the event occurrence information may be missed, due to the limitation of the observation period or track procedure [41], which is called censorship. A key technique addressing censorship is to estimate the probability of the event occurrence at each time typically studied by survival analysis. There are two main streams of survival analysis. The first view is based on traditional statistics scattering in three categories. Non-parametric methods [5, 22] are solely based on counting statistics. Semi-parametric methods [8, 37] assume some base distribution functions with the scaling coefficients. Parametric models [26] assume that the survival time or its logarithm result follows a particular theoretical distribution. These methods either base on statistical counting information or pre-assume distributional forms for the survival rate function, which generalizes not very well in real-world situations. The second school of survival analysis takes from machine learning perspective. These machine learning methodologies include survival random forest [13], Bayesian models [30], support vector machine [25] and multi-task learning solutions [4, 28]. Recently, Ren et al. [32, 33] proposed a recurrent neural network model which captures the sequential dependency patterns between neighboring time slices and estimates the survival rate through the probability chain rule. In this paper, we extend the methodology of the survival analysis to provide fine-grained unbiased ranking list in a unified learning objective without making any distributional assumptions.

3 PRELIMINARY

3.1 Point-wise Unbiased Learning-to-Rank

The fundamental task in learning-to-rank scenarios is to learn a ranker f which assigns a score r to the document d according to item feature $\mathbf{x}$. Then, the documents with respect to the query q return the list in descending order of their scores. Let q denotes the query and D_q the set of documents associated with q . We consider three subsets contained in the set of documents D_q as follows:

- D_q^+ : set of the real positive documents that user has expressed her feedback on, *i.e.*, observed and clicked.
- D_q^- : set of the real negative documents that user has seen but not given her feedback on, *i.e.*, observed and unclicked.
- D_q^* : set of the uncertain documents that user feedback is unclear due to user's leave behavior, *i.e.*, untrusted.

We describe d_i the i -th document in D_q and $\mathbf{x}_i$ the feature vector of d_i . Let r_i represent the relevance of d_i . For simplicity we only consider binary relevance here, *i.e.*, $r_i = 1$, $r_i = 0$ and $r_i = ?$, where $? \in \{0, 1\}$ but unknown for those untrusted documents; and one can easily extend it to the multi-level relevance case. In the point-wise setting, the risk function in learning is defined on a single data point $\mathbf{x}$ as

$$R(f) = \int_q \int_{d_i \in D_q^+} L(f(\mathbf{x}_i), r_i) dP(\mathbf{x}_i, r_i), \quad (1)$$

where f denotes a ranker, $L(f(\mathbf{x}_i), r_i)$ denotes a point-wise loss function and $P(\mathbf{x}_i, r_i)$ denotes the probability distribution on $\mathbf{x}_i$ and r_i . The goal of learning-to-rank is to find the optimal ranker f that minimizes the loss function. Traditionally, the ranker is learned with labeled data containing user browsing logs. However, click data is indicative of individual users' relevance judgments, but is also noisy and biased [2, 21]. This is what we call biased learning-to-rank.

3.2 Pair-wise Unbiased Learning-to-Rank

Traditionally, in the pairwise setting, the ranker f is still defined on a query document pair $(\mathbf{x}, r)$, and the loss function is defined on two data points: positive document d_i and negative document d_j . We here also take those untrusted documents into consideration. Specially, we sample d_i and d_j from three candidate sets, *i.e.*, D_q^+ , D_q^- and D_q^* instead of only from the first two sets. Let q denote a query. Let $\mathbf{x}_i$ and $\mathbf{x}_j$ denote the feature vectors from d_i and d_j respectively. Let r_i and r_j represent the document d_i and document d_j respectively. Let I_q denote the set of document pairs (d_i, d_j) . Similarly, for simplicity we only consider binary relevance here. The risk function is defined as

$$R(f) = \int_q \int_{(d_i, d_j) \in I_q} L(f(\mathbf{x}_i), r_i, f(\mathbf{x}_j), r_j) dP(\mathbf{x}_i, r_i, \mathbf{x}_j, r_j), \quad (2)$$

where $L(f(\mathbf{x}_i), r_i, f(\mathbf{x}_j), r_j)$ denotes a pair-wise loss function.

3.3 Problem Formulation

The key question in any unbiased learning-to-rank algorithm is how to fill the gap between click and relevance. Given the fact that users click a search document d_i ($c_i = 1$) only when it is both observed ($o_i = 1$) and perceived as relevant ($r_i = 1$), most recent works [2, 21, 42, 43] formulate the problem as $P(c_i = 1|\mathbf{x}) = P(c_i =$

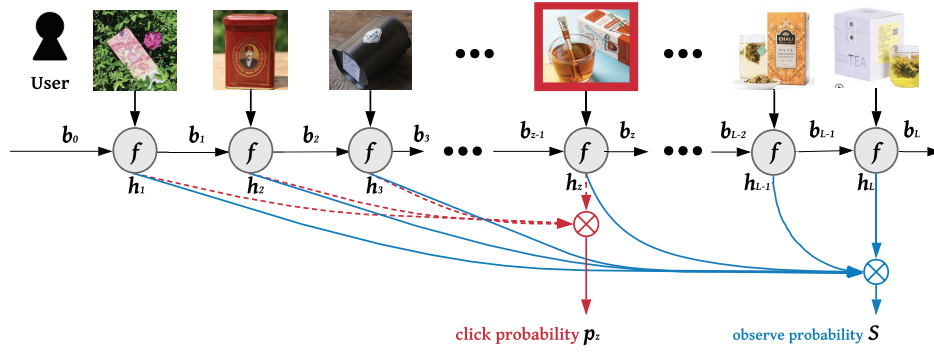


Figure 2: Illustration of Deep Recurrent Survival Ranking model. Note that we mine click patterns in click case and observe patterns in both click and non-click cases.

$1|o_i = 1; \mathbf{x}) \cdot P(o_i = 1|\mathbf{x})$, where we can define relevance as

$$P(r_i = 1|\mathbf{x}) \doteq P(c_i = 1|o_i = 1; \mathbf{x}) = \frac{P(c_i = 1|\mathbf{x})}{P(o_i = 1|\mathbf{x})}. \quad (3)$$

The task of biased learning-to-rank is to estimate click and return a ranking list according to $P(c_i = 1|\mathbf{x})$, while the aim of unbiased learning to rank is to derive relevance from click data and provide a ranking list according to $P(r_i = 1|\mathbf{x})$.

Many click models [7, 10, 15] have investigated how to model the impact from previous clicked document. To simplify, we only study the session with single click here. One can easily extend into multiple click session via truncating multiple one into several single ones. Actually, this sequence truncation method, over the sequential data with multiple events, has been widely used in many works covering various fields such as recommender system [19], conversion attribution [31] and survival analysis [32], which truncates the raw sequences according to the events of interest (i.e., click in our case).

4 METHODOLOGY

4.1 Survival Model

In the field of survival analysis [28, 32], we investigate the probability of death event z happening at each time. Analogously, we here investigate the probability of click event z happening at each document. Let $z = i$ denote that event z happens at i -th document d_i , and $z \geq i$ denote that event z happens after i -th document d_i . We then analyze the patient's investigation on underlying survival period, where a patient will keep 'survival' until she leaves hospital or meets 'death'. Actually, user behaviors on browsing are very similar, where a user will keep observing until she leaves due to lost of interest or clicks due to success in finding a worthwhile document. Hence, we find that click at each item corresponds to the 'death' status of one patient [46], and define click probability, the probability density function (P.D.F.) of click occurring at i -th document d_i , as

$$p_i \doteq P(c_i = 1) \doteq P(z = i), \quad (4)$$

where z denotes the position of clicked document. Also, we see that observe at each item corresponds to the 'survival' status of one patent [46]. Hence, we can derive the observe probability at i -th document d_i as the cumulative distribution function (C.D.F.), since

user will keep browsing until she finds and clicks a favored one, as

$$S(i) \doteq P(o_i = 1) \doteq P(z \geq i) = \sum_{\tau \geq i} P(z = \tau), \quad (5)$$

which represents the probability of the click event occurring after document d_i , i.e., probability of observing d_i . Then it's straightforward to define the *unobserve probability*, i.e., the probability of event occurring before the document d_i , as

$$W(i) \doteq P(o_i = 0) \doteq P(z < i) = \sum_{\tau < i} P(z = \tau). \quad (6)$$

Hence, click probability function at the i -th document can be calculated as

$$\begin{aligned} p_i &= P(z = i) = W(i + 1) - W(i) \\ &= [1 - S(i + 1)] - [1 - S(i)] \\ &= S(i) - S(i + 1). \end{aligned} \quad (7)$$

We define the *relevance probability* as *conditional click probability* according to Eq. (3), the click probability at document d_i given that the previous document d_{i-1} is observed, as

$$h_i \doteq P(r_i = 1) = \frac{P(c_i = 1)}{P(o_i = 1)} = \frac{P(z = i)}{P(z \geq i)} = \frac{p_i}{S(i)}, \quad (8)$$

which also means the probability that the click occurring document z lies at d_i given the condition that z is larger than the last observation boundary.

For those non-click logs caused by user *leave* behavior, we assume that user's favored document (i.e., *click*) hides in the future session. A similar scenario can be found in survival analysis when a patient *leaves* hospital and finally meets 'death' sometime after investigation period. Hence, we can regard these non-click logs as the censored clicked queries where censorship occurs in click. Note that the data logs of unbiased learning-to-rank are represented as a set of triple $\{(\mathbf{x}, z, l)\}$, where $\mathbf{x}$ is the feature of the item and l is the browse length. Here z is the position of clicked document d_z if the user clicks in this browsing behavior, but z is unknown (and we marked z as null) in those non-click browsing histories. Different from traditional causality models [7, 9], survival model is able to capture observe patterns in both click and non-click queries.

4.2 Deep Recurrent Survival Ranking Model

Based on survival model, we introduce our DRSR based on recurrent neural network f_θ with the parameter θ , which captures the sequential patterns for *conditional click probability* h_i at every document

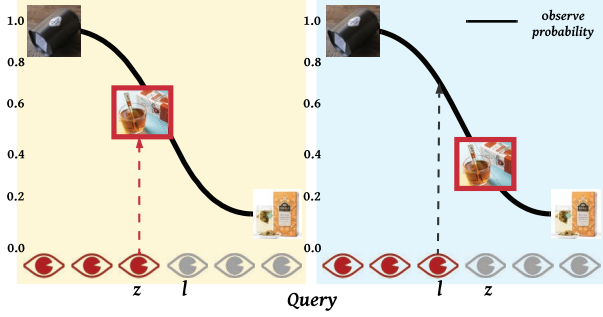


Figure 3: Intuition behind C.D.F. losses. The left and right sub-figures denote click and non-click cases respectively.

d_i . This structure also enables DRSR to take contextual information (i.e., observed documents) into consideration. The detailed structure of DRSR is illustrated in Figure 2. At each document d_i , the i -th RNN cell predicts the *conditional click probability* h_i given the document feature $\mathbf{x}_i$ as

$$h_i = P(z = i \mid z \geq i, \mathbf{x}; \theta) = f_\theta(\mathbf{x}_i \mid b_{i-1}), \quad (9)$$

where f_θ is the RNN function taking $\mathbf{x}$ as input and h_i as output. b_{i-1} is the hidden vector calculated from the last RNN cell. In our paper we implement the RNN function as a standard LSTM unit [16], which has been widely used in sequence data modeling.

From Eqs. (6), (5), (8) and (9), we can easily derive the *observe probability* function $W(i)$ and the *unobserve probability* function $S(i)$ as

$$\begin{aligned} S(i|\mathbf{x}; \theta) &= P(i \leq z|\mathbf{x}; \theta) = P(z \neq 1, z \neq 2, \dots, z \neq i-1|\mathbf{x}; \theta) \\ &= P(z \neq 1|\mathbf{x}_1; \theta) \cdot P(z \neq 2|z \neq 1, \mathbf{x}_2; \theta) \cdots \\ &\quad \cdot P(z \neq i-1|z \neq 1, \dots, z \neq i-2, \mathbf{x}_{i-1}; \theta) \\ &= \prod_{\tau: \tau < i} [1 - P(z = \tau|z \geq \tau, \mathbf{x}_\tau; \theta)] = \prod_{\tau: \tau < i} (1 - h_\tau), \end{aligned} \quad (10)$$

$$W(i|\mathbf{x}; \theta) = P(i > z|\mathbf{x}; \theta) = 1 - S(i|\mathbf{x}; \theta) = 1 - \prod_{\tau: \tau < i} (1 - h_\tau). \quad (11)$$

Here we use probability chain rule to calculate *unobserve probability* $S(i)$ at the given document d_i through multiplying the *conditional unclick probability* $(1 - h_\tau)$, i.e., inverse of the conditional click probability.

Moreover, taking Eqs. (7) and (8) into consideration, the probability of clicked document d_z directly lying at d_i , i.e., *click probability* at document d_i can be written as

$$p_i = P(z = i|\mathbf{x}; \theta) = h_i \prod_{\tau: \tau < i} (1 - h_\tau). \quad (12)$$

4.3 Point-wise Loss Function

In point-wise setting, since there is no ground truth of either event probability distribution or relevance information, here we maximize the log-likelihood over the empirical data distribution to learn our deep model.

The first type of loss is based on the *click probability* (P.D.F.) and it aims to minimize negative log-likelihood of the click document

d_j over the clicked logs as

$$\begin{aligned} L_{\text{point}}(z) &= -\log \prod_{(\mathbf{x}, z) \in \mathcal{D}_{\text{click}}} P(z = j|\mathbf{x}; \theta) = -\log \prod_{(\mathbf{x}, z) \in \mathcal{D}_{\text{click}}} p_j \\ &= -\log \prod_{(\mathbf{x}, z) \in \mathcal{D}_{\text{click}}} [h_j \prod_{\tau: \tau < i} (1 - h_\tau)] \\ &= - \sum_{(\mathbf{x}, z) \in \mathcal{D}_{\text{click}}} [\log h_j + \sum_{\tau: \tau < i} \log(1 - h_\tau)], \end{aligned} \quad (13)$$

where j is the position of true clicked document d_j given the feature vector $\mathbf{x}$.

The second type of loss is based on the *observe probability* (C.D.F.). There are two motivations about the second loss corresponding to these two cases. Let z and l represent clicked document position and browse length respectively. As is shown in Figure 3, the left sub-figure is the click case where z has been known and $z \leq l$; The right sub-figure is the non-click case where z is unknown (censored) but we only have the knowledge that $z > l$.

For the click cases as the left part of Figure 3, we need to “push up” the *observe probability* for the document whose position is in range of $[0, l]$, while “pull down” the *observe probability* for the document whose position is in range of $[l, \infty)$. Thus, on one hand, we adopt the loss over the click cases that

$$\begin{aligned} L_{\text{click}} &= -\log \prod_{(\mathbf{x}, l) \in \mathcal{D}_{\text{click}}} P(l \geq z|\mathbf{x}; \theta) \\ &\approx -\log \prod_{(\mathbf{x}, l) \in \mathcal{D}_{\text{click}}} W(l|\mathbf{x}; \theta) \\ &= - \sum_{(\mathbf{x}, l) \in \mathcal{D}_{\text{click}}} \log [1 - \prod_{\tau: \tau < l} (1 - h_\tau)]. \end{aligned} \quad (14)$$

As for the non-click cases in the right part of Figure 3, we just need to “push up” the *observe probability* since we have no idea about true click document but we only know that $z > l$. On the other hand, we just adopt the loss over the non-click dataset as

$$\begin{aligned} L_{\text{non-click}} &= -\log \prod_{(\mathbf{x}, l) \in \mathcal{D}_{\text{non-click}}} P(z > l|\mathbf{x}; \theta) \\ &\approx -\log \prod_{(\mathbf{x}, l) \in \mathcal{D}_{\text{non-click}}} S(l|\mathbf{x}; \theta) \\ &= - \sum_{(\mathbf{x}, l) \in \mathcal{D}_{\text{non-click}}} \sum_{\tau: \tau < l} \log (1 - h_\tau). \end{aligned} \quad (15)$$

4.4 Permutation Document Model

Different from traditional pair-wise methods where binary classification accompanied with logistic regression is proposed to model relative relevance, as Figure 4 shows, we here model the relative relevance via three conditional probabilities: *click probability* (P.D.F.): (i) $P(z = j|z \geq i)$ for positive document d_j ; (ii) $P(z = i|z \geq j)$ for negative document d_i ; and *observe probability* (C.D.F.): (iii) $P(z \geq k|z \geq i)$ for untrusted document d_k . As Figure 4 shows, the first one indicates the probability of user clicking document d_j given she has browsed document d_i ; the second one represents how likely user click document d_i given she has observed document d_j ; while the third one means the probability of user going on browsing document d_k after observing document d_i .

It should be noted that only the first conditional probability is accessible to be measured since we can only obtain original order 0 (o1 in Figure 4). In order to get rerank 1 (r1) and rerank 2 (r2), we need to permute the documents. Recall that users often browse from the top to bottom, which may result in that higher the document ranked, more likely it to be clicked. We are able to move clicked document d_j forward since it will not change click behavior. By this way, we obtain r1. Considering there may exist relevant documents in those untrusted observations, we consider to move untrusted document d_k forward to get r2. Note that moving d_k forward and d_j backward may change click behavior, we here model observe probability instead. This technique to mine more potential order based on an original order is inspired by XLNet [44], so we call it permutation document modeling.

By this way, our model is able to (i) consider documents' display order in pair-wise setting by modeling relative relevance with conditional probability; (ii) take both trusted and untrusted observation into consideration; (iii) conduct training procedure with more dependency pairs.



Figure 4: Illustration of permutation document modeling, where documents here are sampled from different subsets in the right side. Double sided arrow denotes exchange or order operation.

4.5 Pair-wise Loss Functions

Different from point-wise loss functions, pair-wise loss functions can preserve relative information, *e.g.*, relative relevance can be drawn from user's action to document d_j given she has browsed document d_i . There are three pair-wise loss functions for positive, negative and untrusted documents respectively based on the analysis in Section 4.4.

In order to maximize log-likelihood of *click probability* (P.D.F.): $P(z = j|z \geq i)$, *i.e.*, clicking relevant document d_j after observing irrelevant document d_i , we formulate the first pair-wise loss based on o0 query in Figure 4 as

$$\begin{aligned}
 L_{\text{pair}(o_0)} &= -\log \prod_{(d_i, d_j) \in I_q} P(z = j|z \geq i, \mathbf{x}; \theta) \\
 &= -\log \prod_{(d_i, d_j) \in I_q} \frac{P(z = j|\mathbf{x}; \theta)}{P(z \geq i|\mathbf{x}; \theta)} = -\log \prod_{(d_i, d_j) \in I_q} \frac{p_j}{S(i|\mathbf{x}; \theta)} \\
 &= - \sum_{(d_i, d_j) \in I_q} \{[\log h_j \sum_{\tau: \tau < j} \log(1 - h_\tau)] - \sum_{\tau: \tau < i} \log(1 - h_\tau)\}.
 \end{aligned} \tag{16}$$

In r1 query, we need to minimize log-likelihood of *click probability* (P.D.F.): $P(z = i|z \geq j)$, *i.e.*, clicking irrelevant document d_i after observing relevant document d_j and form the second pair-wise loss function as

$$\begin{aligned}
 L_{\text{pair}(r_1)} &= \log \prod_{(d_i, d_j) \in I_q} P(z = i|z \geq j, \mathbf{x}; \theta) \\
 &= \log \prod_{(d_i, d_j) \in I_q} \frac{P(z = i|\mathbf{x}; \theta)}{P(z \geq j|\mathbf{x}; \theta)} = \log \prod_{(d_i, d_j) \in I_q} \frac{p_i}{S(j|\mathbf{x}; \theta)} \\
 &= \sum_{(d_i, d_j) \in I_q} \{[\log h_i \sum_{\tau: \tau < i} \log(1 - h_\tau)] - \sum_{\tau: \tau < j} \log(1 - h_\tau)\}.
 \end{aligned} \tag{17}$$

For r2 query, we measure the relative relevance between trusted (*i.e.*, relevant and irrelevant) and untrusted documents via user observe behavior. Specially, we evaluate *observe probability* (C.D.F.): $P(z \geq k|z \geq i)$, *i.e.*, probability of user going browsing d_k after she has observed d_i as

$$\begin{aligned}
 L_{\text{pair}(r_2)} &= -\log \prod_{(d_i, d_k) \in I_q} P(z \geq k|z \geq i, \mathbf{x}; \theta) \\
 &= -\log \prod_{(d_i, d_k) \in I_q} \frac{P(z \geq k|\mathbf{x}; \theta)}{P(z \geq i|\mathbf{x}; \theta)} = -\log \prod_{(d_i, d_k) \in I_q} \frac{S(k|\mathbf{x}; \theta)}{S(i|\mathbf{x}; \theta)} \\
 &= - \sum_{(d_i, d_k) \in I_q} [\sum_{\tau: \tau < k} \log(1 - h_\tau) - \sum_{\tau: \tau < i} \log(1 - h_\tau)].
 \end{aligned} \tag{18}$$

4.6 Model Realization

In this section, we unscramble some intrinsic properties of our deep model and analyze the model efficiency in this section.

Properties of Loss Function. First of all, we take the view of click prediction of our methodology. As is known that there is a click status, *i.e.*, an indicator of click event, for each sample as

$$\omega = \begin{cases} 1 & \text{if } l \geq z \\ 0 & \text{otherwise } l < z. \end{cases} \tag{19}$$

Hence, taking Eqs. (14) and (15) altogether and we may find that combination of L_{click} and $L_{\text{non-click}}$ describes the classification of click status at document d_l of each sample as

$$\begin{aligned}
 L_2 &= L_{\text{click}} + L_{\text{non-click}} \\
 &= -\log \prod_{(\mathbf{x}, l) \in \mathcal{D}_{\text{click}}} P(l \geq z|\mathbf{x}; \theta) - \log \prod_{(\mathbf{x}, l) \in \mathcal{D}_{\text{non-click}}} P(z > l|\mathbf{x}; \theta) \\
 &\approx -\log \prod_{(\mathbf{x}, l) \in \mathcal{D}} [W(l|\mathbf{x}; \theta)]^\omega \cdot [1 - W(l|\mathbf{x}; \theta)]^{1-\omega} \\
 &= - \sum_{(\mathbf{x}, l) \in \mathcal{D}} \{\omega \cdot \log W(l|\mathbf{x}; \theta) + (1 - \omega) \cdot \log [1 - W(l|\mathbf{x}; \theta)]\},
 \end{aligned} \tag{20}$$

which is the cross entropy loss for predicting click status at time t given $\mathbf{x}$ over all the data $\mathcal{D} = \mathcal{D}_{\text{click}} \cup \mathcal{D}_{\text{non-click}}$.

Combining all the objective functions and our goal is to minimize the negative log-likelihood over all the data samples including both

clicked and non-click data as

$$\arg \min_{\theta} \alpha L_1 + (1 - \alpha) L_2$$

$$\text{where } L_1 = \begin{cases} L_{\text{point}(z)} & \text{point-wise} \\ L_{\text{pair}(o_0)} + L_{\text{pair}(r_1)} + L_{\text{pair}(r_2)} & \text{pair-wise,} \end{cases} \quad (21)$$

where the hyper-parameter α controls the order of magnitudes of the gradients from the two losses at the same level to stabilize the model training.

In the traditional and related works, they usually adopt only L_1 based on *click probability* (P.D.F.) in unbiased learning-to-rank field [17] for click prediction or $L_{\text{non-click}}$ in survival analysis field [8, 23] for censorship handling. We propose a comprehensive loss function which learns from both click logs and non-click logs. From the discussion above, L_{click} and $L_{\text{non-click}}$ collaboratively learns the data distribution from the *observe probability* (C.D.F.) view.

Model Efficiency. As shown in Eq. (9), each recurrent unit f_{θ} takes $(\mathbf{x}, z_l, b_{l-1})$ as input and outputs probability scalar h_l and hidden vector b_l to the next unit. Let L be the maximal browse length, so the calculation of the recurrent units will run for maximal L times. We assume the average case time performance of recurrent units f_{θ} is $O(C)$, which is related to the implementation of the unit [45], recurrent skip coefficients, yet can be paralleled through GPU processor. The subsequent calculation is to obtain the multiplication results of h_l or $(1 - h_l)$ to get the results of p_z and S , as that in Figure 2, whose complexity is $O(L)$. Thus the overall time complexity is $O(CL) + O(L) = O(CL)$, which is the same as the original recurrent neural network model.

4.7 Learning Algorithm

We provide the learning algorithm of DRSR in Algorithm 1. It should be noted that we calculate loss function $L(\theta)$ with $P(c = 1)$ in training procedure since only click data is available; while we use $P(r = 1)$ in inference procedure, which is actually the core of unbiased learning-to-rank.

Algorithm 1 Deep Recurrent Survival Ranking (DRSR)

Require: dataset $\mathcal{D} = \mathcal{D}_{\text{click}} \cup \mathcal{D}_{\text{non-click}}$;

Ensure: unbiased ranker f_{θ} with parameter θ

- 1: Initialize all parameters.
 - 2: **repeat**
 - 3: Randomly sample a batch $\mathcal{B}$ from $\mathcal{D}$
 - 4: **for** each point d_q or pair $(d_i, d_j) \in I_q$ in $\mathcal{B}$ **do**
 - 5: Calculate $P(r = 1)$ and $P(c = 1)$ using Eqs. (9) and (12).
 - 6: **end for**
 - 7: Compute corresponding $L(\theta)$ according to Eq. (21).
 - 8: Update θ by minimizing $L(\theta)$.
 - 9: **until** convergence
-

5 EXPERIMENTS

In this section, we present the experimental setup and the corresponding results under various evaluation metrics. Furthermore, we look deeper into our model and analyze some insights of the experiment results. Moreover, we have also published our code¹. We start with three research questions (RQ) to lead the experiments and the following discussions.

¹Reproducible code link: <https://github.com/Jinjiarui/DRSR>.

- **(RQ1)** Compared with the baseline models, does DRSR achieve state-of-the-art performance in unbiased learning-to-rank?
- **(RQ2)** Are debiased method, *i.e.*, survival model, truly necessary for improving performance?
- **(RQ3)** Can DRSR learn robustly under different situation, *i.e.*, simulation, bias degree and number of data?

5.1 Datasets and Experiment Flow

We conduct our experiment on Yahoo search engine dataset named Yahoo! learning-to-rank challenge dataset and Alibaba recommender system dataset. We choose NDCG at position 1, 3, 5 and MAP as evaluation measures in relevance ranking.

Data Description. Two large-scale real-world datasets² are used in our experiments.

- **Yahoo search engine dataset** is one of the largest benchmark dataset widely used in unbiased learning-to-rank [2, 17]. It consists of 29,921 queries and 710k documents. Each query document pair is represented by a 700-dimensional feature vector manually assigned with a label denoting relevance at 5 levels [6].
- **Alibaba recommender system dataset** is a proprietary dataset where we regard each user and item information as query and document feature respectively. Similar approaches can be found in [40, 47]. It contains 178,839 queries and 3,133k documents. Each query document pair is represented by a 1750-dimensional feature vector and assigned with a binary label denoting relevance.

Click Data Generation. The click data generation process in [2, 17] is followed. First, one trains a Rank SVM model using 1% of the training data with relevance labels. Next, one uses trained model to create an initial ranking list for each query. Then, one simulates user browsing process and samples clicks from the initial list. We utilize two simulation model here:

- **PBM** [34] simulates this user browsing behavior based on the assumption that the bias of a document only depends on its position, which can be formulated as $P(o_i) = \rho_i^{\tau}$, where ρ_i represents position bias at position i and $\tau \in [0, +\infty]$ is a parameter controlling the degree of position bias. The position bias ρ_i is obtained from an eye-tracking experiment in [20] and the parameter τ is set as 1 by default. It also assumes that a user decides to click a document d_i according to probability $P(c_i) = P(o_i) \cdot P(r_i)$.
- **CCM** [14] is a cascade model, which assumes that the user browses the search results in a sequential order from top to bottom. The user browse behaviors are both conditioned on current and past documents, as $P(c_i = 1 | o_i = 0) = 0$, $P(c_i = 1 | o_i = 1, r_i) = P(r_i)$, $P(o_{i+1} = 1 | o_i = 0) = 0$, $P(o_{i+1} = 1 | o_i = 1, c_i = 0) = \gamma_1$, $P(o_{i+1} = 1 | o_i = 1, c_i = 1, r_i) = \gamma_2 \cdot (1 - P(r_i)) + \gamma_3 \cdot P(r_i)$. The parameter is obtained from experiment in [14] with $\gamma_2 = 0.10$ and $\gamma_3 = 0.04$ for navigational queries (Yahoo search engine); $\gamma_2 = 0.40$ and $\gamma_3 = 0.27$ for informational queries (Alibaba recommender system). γ_1 is set as 0.5 by default.

The probability of relevance $P(r_i)$ is calculated by $P(r_i) = \epsilon + (1 - \epsilon) \cdot \frac{2^{y_i} - 1}{2^{y_{\max}} - 1}$, where $y_i \in [0, 4]$ represents relevance level. The parameter ϵ denotes click noise and is set as 0.1 as default.

²Dataset download link: <http://webscope.sandbox.yahoo.com>.

Table 2: Comparison of different unbiased learning-to-rank methods under Yahoo Search Engine and Alibaba Recommender System. CCM is utilized as click generation model. * indicates p-value < 0.001 in significance test vs the best baseline.

Ranker	Debiasing Method	Yahoo Search Engine (CCM)				Alibaba Recommender System (CCM)			
		MAP	NDCG@1	NDCG@3	NDCG@5	MAP	NDCG@1	NDCG@3	NDCG@5
DRSR (Ours)	Labeled Data	0.861	0.747	0.759	0.771	0.850	0.737	0.741	0.755
	Pairwise Debiasing	0.842*	0.719*	0.721*	0.737*	0.831*	0.684*	0.685*	0.707*
	Pointwise Debiasing	0.839*	0.713*	0.717*	0.730*	0.830*	0.682*	0.684*	0.706*
	Regression-EM [43]	0.829	0.679	0.685	0.701	0.820	0.657	0.668	0.673
	Click Data	0.817	0.636	0.652	0.667	0.810	0.613	0.627	0.658
LambdaMART	Labeled Data	0.854	0.745	0.745	0.757	0.847	0.729	0.732	0.743
	Ratio Debiasing [17]	0.830	0.688	0.685	0.699	0.821	0.661	0.669	0.674
	Regression-EM [43]	0.826	0.669	0.676	0.691	0.818	0.636	0.651	0.667
	Click Data	0.813	0.628	0.646	0.673	0.804	0.603	0.618	0.646
DNN	Labeled Data	0.831	0.677	0.685	0.705	0.824	0.674	0.679	0.693
	Dual Learning Algorithm [2]	0.825	0.672	0.678	0.691	0.814	0.629	0.647	0.674
	Regression-EM [43]	0.823	0.665	0.669	0.687	0.813	0.628	0.645	0.672
	Click Data	0.809	0.611	0.619	0.648	0.801	0.600	0.612	0.641

5.2 Compared Settings

We made comprehensive comparisons between our model and the baselines. The baselines are created by combining the learning-to-rank algorithm with the state-of-the-art debiasing methods.

- **Regression-EM:** Wang et al. [43] proposed regression-based EM method where position bias is estimated directly from regular production clicks.
- **Dual Learning Algorithm:** Ai et al. [2] proposed a dual learning which can jointly learn a ranker and conduct debiasing of click data.
- **Ratio Debiasing:** Hu et al. [17] proposed an unbiased pair-wise learning-to-rank based on inverse propensity weight (IPW) [42].
- **Point-wise Debiasing:** Our proposed debiasing method DRSR which is adapted in point-wise setting.
- **Pair-wise Debiasing:** Our proposed debiasing method DRSR which is adapted in pair-wise setting.
- **Click Data:** We utilize the raw click data without debiasing to train the ranker, whose performance is regarded as a lower bound.
- **Labeled Data:** The human annotated relevance labels without any bias are used as data for training the ranker and we consider its performance as an upper bound.

There are several learning-to-rank algorithms cooperated with debiasing methods.

- **DRSR:** Our model.
- **DNN:** A deep neural network as described in [2] is implemented as a ranker.
- **LambdaMART:** We implement Unbiased LambdaMART by modifying the LambdaMART tool in LightGBM [24].

In summary, there are 11 baselines to compare with our model. Note that Dual Learning and DNN are tightly coupled. Also, the same situation happens in Ratio Debiasing and LambdaMART. We don't combine Ratio Debiasing with DNN and Dual Learning with LambdaMART, as it's beyond the scope of this paper.

5.3 Result Analysis

Experimental Results and Analysis (RQ1). Table 2 summarizes the results. We see that our method of Deep Recurrent Survival Ranking Models (DRSR + Pointwise Debiasing / Pairwise Debiasing) significantly outperform all the other baseline methods. The

results of Ratio Debiasing, Regression-EM and Dual Learning Algorithm are comparable with those reported in the original papers. In particular, we have the following findings:

- Our models based DRSR achieve better performances than all the state-of-the-art methods in terms of all measures, which indicates our framework DRSR outperforms other models such as LambdaMART and DNN. The reason seems to be that our framework enables to find correlation of user various behaviors and mine hidden observe pattern in non-click queries.
- Pairwise Debiasing works better than the other debiasing methods when combined with DRSR framework. This implies that considering relative relations between trusted and untrusted observation can enhance model performance.
- When conducted in Alibaba Recommender System, the performances of all models decrease significantly. This implies that items in recommendation are in a larger scale and have a more complex feature space than in search engine.
- The performances of Pairwise Debiasing and Pointwise Debiasing get closer in Alibaba Recommender System. This indicates that it is challenging to define and capture relative relevance in recommendation, since various items in different categories can be displayed at the same time. Also, the user preference is more personalized, dynamic and even noisy.

Ablation Study (RQ2). In order to analyze the importance of survival model (debiasing method) and recurrent neural network (ranker), we also conduct experiment of recurrent neural network without survival model and summarize results as Click Data of DRSR in Table 2. We can find that sophisticated algorithms like DRSR and LambdaMART are sensitive to position bias when comparing the performance of Click Data with Debiasing methods, which indicates the significance for unbiased learning-to-rank. Also, we can see that when trained with human labeled data, DRSR achieves the best performance (Labeled Data) which implies that there is still much room for improvement in unbiased learning-to-rank.

Visualization Analysis (RQ2). We investigated whether the performance improvement by DRSR is indeed from reduction of position bias through comparing the ranking list given by the initial ranker with debiased ranker.

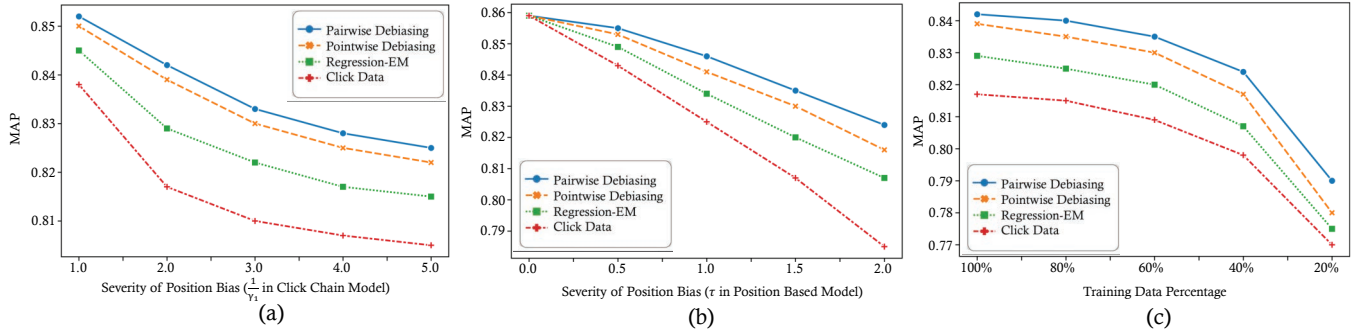


Figure 5: (a) & (b) Performance of DRSR against other debiasing methods with different degrees of position bias. (c) Performance of DRSR against other debiasing methods with different sizes of training data.

We first identified the documents at each position given by the initial ranker. Then, we calculated the average positions of the documents at each original position after re-ranking by various debiasing methods, combined with DRSR. We also calculated the average positions of the documents after re-ranking by their relevance labels, which is regarded as the ground truth. Ideally, the average positions by the debiasing methods should get close to the average position by the relevance labels. We summarized the results and show them in Figure 6.

One can see that the curve of Click Data (in red) is away from that of Relevance Label (in purple), indicating that directly using click data without debiasing can be problematic. The curve of Pairwise Debiasing (in blue) and Pointwise Debiasing (in orange) are the closest to the curve of Relevance Label, representing that the performance enhancement of DRSR is indeed from effective debiasing.

Generalizability Analysis (RQ3). The click data utilized in Table 2 is generated by Click Chain Model (CCM), a cascade model, which assumes that the user browses the search results in a sequential order. One can see that setting of DRSR and CCM match each other well, which may affect the performance. Hence, we need to evaluate DRSR in a more general view. The Position Based Model (PBM) assumes that the bias of a document only depends on its position, which is a general approximation of user click behavior in practice. We compared the same baseline methods here. Again, we found that DRSR significantly outforms the baselines, indicating that our model is indeed an effective method.

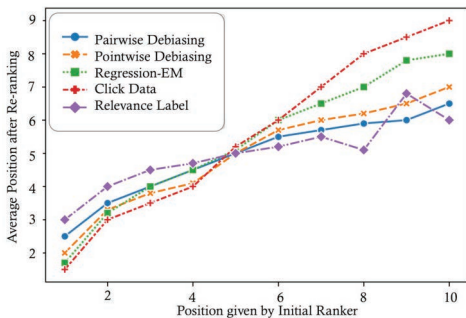


Figure 6: Average positions after re-ranking of documents at each original position by different debiasing methods combined with DRSR.

Table 3: Comparison with PBM as click generation model. Notations are same with Table 2.

Yahoo Search Engine (PBM)				
Ranker	MAP	NDCG@1	NDCG@3	NDCG@5
DRSR (Ours)	0.861	0.747	0.759	0.771
	0.848*	0.726*	0.737*	0.745*
	0.843*	0.723*	0.731*	0.740*
	0.834	0.698	0.705	0.712
	0.825	0.671	0.679	0.693
LambdaMART	0.854	0.745	0.745	0.757
	0.836	0.717	0.716	0.728
	0.830	0.685	0.684	0.700
	0.820	0.658	0.669	0.672
DNN	0.831	0.677	0.685	0.705
	0.828	0.674	0.683	0.697
	0.829	0.676	0.684	0.699
	0.819	0.637	0.651	0.667

Robustness Analysis (RQ3). We further evaluated the robustness of DRSR under different degrees of position bias and different size of training data. In the above experiments, we only tested the performance of DRSR with click data generated from a single click model, *i.e.*, $\gamma_1 = 0.5$ in Click Chain Model and $\tau = 1$ in Position Based Model. Here, γ_1 and τ influence the probability that user exams the next result. Obviously, the smaller γ_1 and larger τ indicate that the user will have a smaller probability to continue reading, which means a more severe position bias. Therefore, here we set the two hyper-parameters to different values and examined whether DRSR can still work equally well.

Figure 5(a) & (b) show the results in terms of MAP with different degrees of position bias. The results in terms of other measures have similar trends. When τ in PBM equals 0, there is no position bias; while γ_1 in CCM equals 1, there still exist position bias brought from γ_2 and γ_3 . The results of all debiasing methods are similar to that of using click data only. As we add more position bias, *i.e.*, τ increases and γ_1 decreases, the performances of all the debiasing methods decrease dramatically. However, under all settings DRSR can get less affected by position bias and consistently maintain the best results. This indicates that DRSR is robust to different degrees of position bias.

Next, we investigated the robustness of DRSR under different sizes of training data. We first randomly selected a subset of training data (*i.e.*, 20% - 100%) to generate different sizes of click datasets, and then used these datasets to evaluate the performances of DRSR

with different debiasing methods. To make fair comparison, we utilized the same subsets of training data for Regression-EM.

As shown in Figure 5(c), when the size of training data decreases, the improvements obtained by the debiasing methods also decrease. The reason seems to be that the position bias estimated from insufficient training data is not accurate, which can hurt the performances of the debiasing methods. DRSR Debiasing which adopts a joint training mechanism, can still achieve the best performances in such cases. Also, Pairwise Debiasing enhances its performance via data augmentation in document permutation model. The experiment shows that DRSR can still work well even with limited training data.

6 CONCLUSION AND FUTURE WORK

In this paper, we propose an innovative framework named DRSR where we adopt survival analysis techniques accompanied with probability chain rule to derive the joint probability of user various behaviors. This framework enables unbiased model to leverage the contextual information in the ranking list to enhance the performance. Also, we incorporate with survival analysis, and thus can model the non-click queries as the censored click logs, where the censorship occurs in click. We design a novel objective function to mine the rich observe and click patterns hidden in both click and non-click queries. Also, we extend pair-wise loss to capture relative relevance between trusted feedback and untrusted observation via conditional probability. In the future work, it would be interesting to investigate better solution to model multiple-click session and consider good and bad in abandoned, *i.e.*, non-click queries, respectively.

Acknowledgments. The co-corresponding authors are Weinan Zhang and Kan Ren. We thank the support of National Natural Science Foundation of China (Grant No. 61702327, 61772333, 61632017) and Wu Wen Jun Honorary Doctoral Scholarship from AI Institute, Shanghai Jiao Tong University.

REFERENCES

- [1] Aman Agarwal, Kenta Takatsu, Ivan Zaitsev, and Thorsten Joachims. 2019. A General Framework for Counterfactual Learning-to-Rank. In *SIGIR*.
- [2] Qingyao Ai, Keping Bi, Cheng Luo, Jiafeng Guo, and W Bruce Croft. 2018. Unbiased Learning to Rank with Unbiased Propensity Estimation. *SIGIR* (2018).
- [3] Qingyao Ai, Jiaxin Mao, Yiqun Liu, and W Bruce Croft. 2018. Unbiased learning to rank: Theory and practice. In *CIKM*.
- [4] Ahmed M Alaa and Mihaela van der Schaar. 2017. Deep multi-task gaussian processes for survival analysis with competing risks. In *NeurIPS*.
- [5] Per K Andersen, Ornulf Borgan, Richard D Gill, and Niels Keiding. 2012. *Statistical models based on counting processes*.
- [6] Olivier Chapelle and Yi Chang. 2011. Yahoo! learning to rank challenge overview. In *Proceedings of the Learning to Rank Challenge*.
- [7] Olivier Chapelle and Ya Zhang. 2009. A dynamic bayesian network click model for web search ranking. In *WWW*.
- [8] David R Cox. 1972. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological)* (1972).
- [9] Nick Craswell, Onno Zoeter, Michael Taylor, and Bill Ramsey. 2008. An experimental comparison of click position-bias models. In *WSDM*.
- [10] Georges E Dupret and Benjamin Piwowarski. 2008. A user browsing model to predict search engine click data from past observations. In *SIGIR*.
- [11] Hui Fang, Guibing Guo, Danning Zhang, and Yiheng Shu. 2019. Deep Learning-Based Sequential Recommender Systems: Concepts, Algorithms, and Evaluations. In *International Conference on Web Engineering*.
- [12] Zhichong Fang, Aman Agarwal, and Thorsten Joachims. 2018. Intervention harvesting for context-dependent examination-bias estimation. *SIGIR* (2018).
- [13] Louis Gordon and Richard A Olshen. 1985. Tree-structured survival analysis. *Cancer treatment reports* (1985).
- [14] Fan Guo, Chao Liu, Anitha Kannan, Tom Minka, Michael Taylor, Yi-Min Wang, and Christos Faloutsos. 2009. Click chain model in web search. In *WWW*.
- [15] Fan Guo, Chao Liu, and Yi Min Wang. 2009. Efficient multiple-click models in web search. In *WSDM*.
- [16] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* (1997).
- [17] Ziniu Hu, Yang Wang, Qu Peng, and Hang Li. 2019. Unbiased LambdaMART: An Unbiased Pairwise Learning-to-Rank Algorithm. In *WWW*.
- [18] Rolf Jagerman, Harrie Oosterhuis, and Maarten de Rijke. 2019. To Model or to Intervene: A Comparison of Counterfactual and Online Learning to Rank from User Interactions. (2019).
- [19] How Jing and Alexander J Smola. 2017. Neural survival recommender. In *WSDM*.
- [20] Thorsten Joachims, Laura A Granka, Bing Pan, Helene Hembrooke, and Geri Gay. 2005. Accurately interpreting clickthrough data as implicit feedback. In *SIGIR*.
- [21] Thorsten Joachims, Adith Swaminathan, and Tobias Schnabel. 2017. Unbiased learning-to-rank with biased feedback. In *WSDM*.
- [22] Edward L Kaplan and Paul Meier. 1958. Nonparametric estimation from incomplete observations. *Journal of the American statistical association* (1958).
- [23] Jared L Katzman, Uri Shaham, Alexander Cloninger, Jonathan Bates, Tingting Jiang, and Yuval Kluger. 2018. DeepSurv: personalized treatment recommender system using a Cox proportional hazards deep neural network. *BMC medical research methodology* (2018).
- [24] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. Lightgbm: A highly efficient gradient boosting decision tree. In *NeurIPS*.
- [25] Faisal M Khan and Valentina Bayer Zubek. 2008. Support vector regression for censored data (SVRC): a novel tool for survival analysis. In *ICDM*.
- [26] Elisa T Lee and John Wang. 2003. *Statistical methods for survival data analysis*. Vol. 476. John Wiley & Sons.
- [27] Jane Li, Scott Huffman, and Akihito Tokuda. 2009. Good abandonment in mobile and PC internet search. In *SIGIR*.
- [28] Yan Li, Jie Wang, Jieping Ye, and Chandan K Reddy. 2016. A multi-task learning formulation for survival analysis. In *KDD*.
- [29] Tie-Yan Liu et al. 2009. Learning to rank for information retrieval. *Foundations and Trends® in Information Retrieval* (2009).
- [30] Rajesh Ranganath, Adler Perotte, Noémie Elhadad, and David Blei. 2016. Deep survival analysis. *arXiv* (2016).
- [31] Kan Ren, Yuchen Fang, Weinan Zhang, Shuhao Liu, Jiajun Li, Ya Zhang, Yong Yu, and Jun Wang. 2018. Learning multi-touch conversion attribution with dual-attention mechanisms for online advertising. In *CIKM*.
- [32] Kan Ren, Jiarui Qin, Lei Zheng, Zhengyu Yang, Weinan Zhang, Lin Qiu, and Yong Yu. 2019. Deep recurrent survival analysis. In *AAAI*.
- [33] Kan Ren, Jiarui Qin, Lei Zheng, Weinan Zhang, and Yong Yu. 2019. Deep Landscape Forecasting for Real-time Bidding Advertising. *KDD* (2019).
- [34] Matthew Richardson, Ewa Dominowska, and Robert Ragno. 2007. Predicting clicks: estimating the click-through rate for new ads. In *WWW*.
- [35] Paul R Rosenbaum and Donald B Rubin. 1983. The central role of the propensity score in observational studies for causal effects. *Biometrika* (1983).
- [36] Yang Song, Xiaolin Shi, Ryan White, and Ahmed Hassan Awadallah. 2014. Context-aware web search abandonment prediction. In *SIGIR*.
- [37] Robert Tibshirani. 1997. The lasso method for variable selection in the Cox model. *Statistics in medicine* (1997).
- [38] Chao Wang, Yiqun Liu, Meng Wang, Ke Zhou, Jian-yun Nie, and Shaoping Ma. 2015. Incorporating non-sequential behavior into click models. In *SIGIR*.
- [39] Hongning Wang, ChengXiang Zhai, Anlei Dong, and Yi Chang. 2013. Content-aware click modeling. In *WWW*.
- [40] Jun Wang, Arjen P De Vries, and Marcel JT Reinders. 2006. A user-item relevance model for log-based collaborative filtering. In *ECIR*.
- [41] Ping Wang, Yan Li, and Chandan K Reddy. 2019. Machine learning for survival analysis: A survey. *CSUR* (2019).
- [42] Xuanhui Wang, Michael Bendersky, Donald Metzler, and Marc Najork. 2016. Learning to rank with selection bias in personal search. In *SIGIR*.
- [43] Xuanhui Wang, Nadav Golbandi, Michael Bendersky, Donald Metzler, and Marc Najork. 2018. Position bias estimation for unbiased learning to rank in personal search. In *WSDM*.
- [44] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V Le. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. *arXiv* (2019).
- [45] Saizheng Zhang, Yuhui Wu, Tong Che, Zhouhan Lin, Roland Memisevic, Ruslan R Salakhutdinov, and Yoshua Bengio. 2016. Architectural complexity measures of recurrent neural networks. In *NeurIPS*.
- [46] Weinan Zhang, Tianxiong Zhou, Jun Wang, and Jian Xu. 2016. Bid-aware Gradient Descent for Unbiased Learning with Censored Data in Display Advertising. In *KDD*.
- [47] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning Tree-based Deep Model for Recommender Systems. In *KDD*.